Terminal-Scalar-Aware Compilation of Quadratic Scores for CKKS

Leonard van Hemert

19 October 2026

Abstract

An encrypted scalar quadratic score need not preserve the intermediate matrix–vector product used to evaluate it. We study a compiler pass for public symmetric quadratic forms that chooses A with $A+A^\top = 2Q$ before applying existing CKKS linear-transform kernels. Orienting opposite cyclic offsets gives at most $\lfloor n/2 \rfloor + 1$ active diagonals in every positive dimension. We prove a matching lower bound among these representatives when every cyclic diagonal of Q contains a nonzero entry. This is a support optimum, not a ciphertext-circuit or runtime optimum. The implementation uses pinned Lattigo kernels and compares symmetric, triangular, direct, BSGS, and additional quadratic baselines. In separate client/server processes at batch 128, selected plans reduce full loopback inference time by 4.6%, 16.5%, and 14.0% at dimensions 8, 16, and 32, with lower measured server peak memory. A faster dimension-32 plan increases memory and evaluation-key requirements; a dimension-8 BSGS plan is slower. All use the same parameters, security assessment, model, and 10^{-3} absolute-error requirement. Exact rational correspondence checks, encrypted validation, frozen records, and a Lean project expose distinct mathematical, numerical, and implementation verification boundaries. Historical priority for this precise compiler rule remains unestablished.

1 Problem and contribution

Consider a public score

$$f(x) = x^\top Q x + \ell^\top x + c, \qquad Q = Q^\top \in \mathbb{R}^{n\times n}. \tag{1}$$

The client encrypts x, the server computes on ciphertexts, and only the client decrypts the final score. The target includes square-activation neural scores

$$a\left(\sum_{k=1}^{h} v_k (w_k^\top x + b_k)^2 + c_{\text{out}}\right),$$

whose public coefficients can be folded exactly to (1). Folding alone is an essential baseline, not the proposed contribution. The measured domain is $[-1,1]^n$, $n \in \{8,16,32\}$, with 128 independent examples per encrypted request. The construction is general in positive dimension; this particular Go packing/backend supports power-of-two dimensions and the measured dimensions are 8, 16, and 32. The algebraic theorem and Python construction are not restricted to powers of two. We do not claim classifier accuracy on a trained application, a general deep-network speedup, or improved bootstrapping.

A conventional lowering computes Qx, multiplies componentwise by x, and sums. When that vector has no other consumer, it suffices to preserve its scalar contraction. The pass studied here selects A satisfying

$$A + A^\top = 2Q \tag{2}$$

and computes $\sum_i x_i((Ax)_i + \ell_i) + c$. This enlarges the compiler's choice before lowering to existing kernels. A triangular representative already exploits symmetry, but ordinary index-order triangularity can leave every cyclic diagonal active. Orienting *offset pairs*, rather than individual index-ordered pairs, exposes a different sparse linear transform.

The candidate contribution comprises a precise scalar-consumer rewrite, a general restricted support theorem, an implementation correspondence check, and measured interaction with CKKS plans and lifecycle costs. Elementary quadratic identities, skew-symmetric freedom, triangular elimination, diagonal decompositions, BSGS, hoisting, and cryptographic constructions are not claimed as inventions. Nor does the absence of a located matching paper establish historical priority. The release makes a useful mechanism independently inspectable without requiring a priority claim.

2 Threat model and requirements

One client owns the secret key, inputs, and final outputs. The server is honest but curious: it follows the prescribed computation and attempts to infer information from its observations. Model weights, dimensions, batch size, parameters, schedule, timing, memory use, evaluation keys, and serialized lengths are public. The client sends no decrypted intermediate values, decoded outputs, error statistics, or acceptance decisions to the server. Client packing only replicates coordinates; it performs no model-dependent feature extraction. The synthetic benchmark fixtures are deliberately public research evidence, rather than private deployment inputs.

The principal measurement uses distinct processes. The server is an exec-child launched before client keys and fixtures are initialized, receives sanitized public configuration through a pipe, and receives evaluation keys and encrypted requests over HTTP. Its interface has no secret-key/decryptor argument. The client alone decrypts the returned final ciphertext. This process separation is an architectural check, not an alternative cryptographic assumption. Older combined-role experiments are explicitly identified in Section 6.2.

The security target is classical Category 128 under the selected CKKS/RLWE assumptions, assessed using published parameter guidance. We claim neither a new cryptographic proof nor a fresh lattice-estimator run. Passive privacy does not supply malicious-server integrity or circuit privacy; no decryption-feedback protocol is introduced. Every compared method must meet the same 10^{-3} absolute-error threshold against the original neural score, with unchanged model coefficients, no rank truncation, retraining, or activation approximation. Exact algebra and observed CKKS error are assessed separately.

3 Construction and complete support proof

Index rows and columns by $\mathbb{Z}/n\mathbb{Z}$, where $n > 0$, and let $\bar{r} \in \{0, \dots, n-1\}$ denote the canonical integer representative. Define

$$S(M) = \operatorname{supp}_{\mathrm{cyc}}(M) = \{r \in \mathbb{Z}/n\mathbb{Z} : \exists i,\ M_{i,i+r} \neq 0\}.$$

A representative is *admissible* for Q when (2) holds. Call Q *cyclically dense* if for every $r \in \mathbb{Z}/n\mathbb{Z}$ there is at least one i with $Q_{i,i+r} \neq 0$. This is weaker than requiring every entry nonzero, but stronger than requiring some off-diagonal entries nonzero. It is the explicit hypothesis for the lower bound.

Theorem 1 (Scalar preservation)**.** *For real matrices Q, A of equal finite dimension, if $A + A^\top = 2Q$, then $x^\top Ax = x^\top Qx$ for every real vector x. Hence the affine quadratic scores with the same ℓ, c agree everywhere.*

Proof. Let $q_M(x) = \sum_{i,j} x_i M_{ij} x_j$. Exchanging the finite indices i, j and using commutativity gives $q_{A^\top}(x) = q_A(x)$. Multiplying the entrywise hypothesis by $x_i x_j$ and summing gives $2q_A(x) = q_A(x) + q_{A^\top}(x) = 2q_Q(x)$. Division by 2 in $\mathbb{R}$ proves the claim. Adding the identical affine terms preserves equality. Symmetry of Q is unnecessary for this implication (and follows from the existence of an admissible A). □

Let $C_n = \{r : \bar{r} \leq \overline{-r}\}$. The *cyclic orientation* of symmetric Q is

$$O(Q)_{ij} = \begin{cases} Q_{ij}, & r = -r, \\ 2Q_{ij}, & \bar{r} < \overline{-r}, \\ 0, & \bar{r} > \overline{-r}, \end{cases} \qquad r = j - i \in \mathbb{Z}/n\mathbb{Z}. \tag{3}$$

The self-inverse offsets are 0 and, when n is even, $n/2$. They are retained without doubling. Thus C_n consists of offsets 0 through $\lfloor n/2 \rfloor$. This also specifies the cases $n = 1$ and $n = 2$ without exception.

Theorem 2 (Orientation preservation and support)**.** *For every $n > 0$ and symmetric Q, $O(Q) + O(Q)^\top = 2Q$, so the score is preserved for every x. Moreover,*

$$S(O(Q)) \subseteq C_n, \qquad |S(O(Q))| \leq \lfloor n/2 \rfloor + 1.$$

Proof. For a fixed pair (i, j) the transposed entry has offset $-r$. If $r = -r$, both oriented entries equal their symmetric Q entries and sum to $2Q_{ij}$. Otherwise exactly one of $\bar{r} < \overline{-r}$ and its reverse holds. The two entries are therefore $2Q_{ij}$ and 0, whose sum is again $2Q_{ij}$. Theorem 1 applies. Entries outside C_n vanish by definition. For nonzero r, $\overline{-r} = n - \bar{r}$, so membership in C_n is exactly $2\bar{r} \leq n$; the zero offset also belongs. There are $\lfloor n/2 \rfloor + 1$ such integers. This proves the support bound in odd, even, and singleton dimensions. □

Theorem 3 (Matching restricted lower bound)**.** *Let $n > 0$ and let Q be cyclically dense. Every admissible A satisfies*

$$|S(A)| \geq \lfloor n/2 \rfloor + 1.$$

Consequently, when Q is also symmetric, the orientation attains the minimum cyclic-support cardinality among all admissible representatives.

Proof. For each r choose i with $Q_{i,i+r} \neq 0$. Admissibility gives $A_{i,i+r} + A_{i+r,i} = 2Q_{i,i+r} \neq 0$, so at least one of $r, -r$ lies in $S = S(A)$. Thus $S \cup (-S) = \mathbb{Z}/n\mathbb{Z}$. For $r = 0$ the same argument forces $0 \in S$, hence $0 \in S \cap (-S)$. Negation is a bijection, so

$$n = |S \cup (-S)| = 2|S| - |S \cap (-S)| \leq 2|S| - 1.$$

It follows that $|S| \geq \lceil (n+1)/2 \rceil = \lfloor n/2 \rfloor + 1$. In even dimensions the self-inverse half-turn is itself forced into S; the displayed weaker intersection estimate already yields the required integer bound. The upper bound in Theorem 2 matches it. The argument does not hypothesize a lower bound on support and does not restrict A to the particular orientation algorithm. □

The theorem concerns a matrix representation lowered through cyclic diagonals. It does not lower-bound rotations after BSGS, temporary memory, key-switch cost, or arbitrary encrypted circuits. For sparse Q, cyclic density can fail and the stated lower bound need not apply. An implementation should omit exact zero diagonals and compare the original plan as well as the oriented plan.

Compiler rule. The sole-consumer check is a semantic precondition: rewriting Qx would be invalid if another consumer needed that vector. Public folding and admissibility checks precede encrypted execution. A deterministic rule is:

```
Require a terminal real scalar sum(i, x[i] * y[i]);
fold public quadratic coefficients into symmetric Q, ell, c;
for every i,j:  r = (j-i) mod n;
  if r == -r:  A[i,j] = Q[i,j];
  else if value(r) < value(-r):  A[i,j] = 2*Q[i,j];
  else:  A[i,j] = 0;
check exact A + transpose(A) == 2*Q;
lower A using candidate direct/BSGS plans;
retain baseline if the deployment objective prefers it.
```

The Python API rejects nonsymmetric, nonfinite, nonsquare, or overflowing inputs. Its postcondition is checked using exact rational interpretations of stored binary64 numbers. For the frozen dyadic models, public folding has zero coefficient residual. Arbitrary public inputs may have a reported storage residual and must satisfy the same error budget.

4 CKKS realization and deployment cost

We use CKKS [4] through Lattigo v6.2.0 [10]. Standard CKKS at ring degree $N = 8192$ has 4096 complex slots. For batch $B = 128$, feature i of example b is placed in slot $iB + b$, repeated with period nB across all slots. Since nB divides 4096 in the measured dimensions, a rotation by rB implements a cyclic feature shift without mixing examples. The client performs replication only. Matrices Q, triangular representatives, and oriented representatives use this identical layout, encrypted input size, and output size.

The server forms Ax using diagonal plaintexts and the official hoisted direct or BSGS linear-transform evaluator, rescales, and adds ℓ. It drops one level of the unchanged encrypted x, multiplies it componentwise with $Ax + \ell$, relinearizes, rescales, and reduces the n feature blocks using $\log_2 n$ rotations. Adding c finishes the score. Only the resulting ciphertext returns to the client. Both baseline and candidate use one ciphertext–ciphertext multiplication, one relinearization, two rescale levels, and zero bootstraps. The multiplicative depth in variable–variable products is one; two consumed rescale levels should not be confused with two such products.

For a dense model, orientation changes the plaintext diagonal-product count from n to $n/2 + 1$ in the measured even dimensions. It does not reduce ciphertext multiplication depth. Direct lowering generally eliminates input rotations and keys; BSGS can change this relationship because its baby/giant grouping and key set depend on the support. Kernel scheduling and hoisting remain existing backend mechanisms [7]. A support-optimal matrix need not give the fastest or smallest-memory plan.

Parameters. The final Standard configuration uses $\log_2 Q_i$ bit sizes $[58, 40, 40]$, auxiliary modulus bit size $[60]$, scale 2^{40}, uniform ternary secret, and default truncated Gaussian error with $\sigma = 3.2$. The actual primes are

$$Q = [288230376151760897, 1099511480321, 1099511922689],$$
$$P = [1152921504606830593].$$

Their measured $\log_2(QP) \approx 198.0000002$, including the key-switching auxiliary prime. Prime/congruence checks and modulus-bit accounting are part of the evidence validation. The assessment compares this with the published uniform-ternary $N = 8192$ Category-128 bound of 214 bits [3]. This is a published-table assessment under stated assumptions, not a security proof or an unperformed estimator computation. Earlier development used $P = 50$; those smoke results are separate, and final comparisons share $P = 60$. The final modulus-to-scale ratio is about 2^{18}; the archived model's global score bound of 1814 lies within its centered range. The main evidence here uses Standard CKKS. Conjugate-invariant comparisons are secondary matched-batch experiments with their separate ring assumptions; their extra slot capacity is not silently assigned to the candidate.

Lifecycle objective. For a client key reused over H inferences, choose among measured legal plans using

$$F + S_p + HT_p, \qquad K_p \le K_{\max}, \quad M_p \le M_{\max}. \tag{4}$$

Here F is once-per-public-model folding, S_p includes public encoding, key generation/serialization, key distribution and installation, startup and warmup, and T_p is the complete recurring encrypted chain. K_p is serialized evaluation-key size and M_p is peak server RSS. The implementation's profile-based selector is an explicit optimization over candidates, not a request for an LLM to guess cryptographic parameters. Memory and key constraints can choose dimension-32 BSGS over faster direct execution.

Measured HTTP distribution is loopback TCP. For WAN planning one can explicitly add bandwidth and latency estimates to (4), using key and ciphertext byte counts. Such a model is not a measured WAN result; do not double-count measured loopback transport. Setup is neither omitted nor folded into per-inference time without a stated H.

5 Correctness, error, and formal verification

Theorems 1–3 are exact statements over the reals. Public folding, stored coefficients, and CKKS execution introduce separate obligations. For a residual polynomial $x^\top Rx + s^\top x + t$ and $\|x\|_\infty \le R_0$,

$$|x^\top Rx + s^\top x + t| \le R_0^2 \sum_{ij} |R_{ij}| + R_0 \sum_i |s_i| + |t|. \tag{5}$$

This follows term by term from the triangle inequality. The exact rational checker uses this bound for coefficient certificates, including affine square comparators. It does not discard small eigenvalues. For orientation the stored-coefficient residual is exactly zero in the frozen models.

If an input perturbation has $\|e\|_\infty \le \eta$, direct expansion yields

$$|f(x+e) - f(x)| \le (2R_0\eta + \eta^2) \sum_{ij} |Q_{ij}| + \eta \|\ell\|_1.$$

For symmetric Q, orientation preserves the total entrywise coefficient mass: every non-self-inverse pair replaces two equal-magnitude entries by one doubled entry, and self-inverse entries are unchanged. Thus it does not increase this particular global mass bound. This fact is insufficient to bound CKKS error: different key-switching and accumulation schedules still have different numerical behavior.

A conditional execution bound would propagate proven linear-transform, rescale, product, and reduction error bounds. For example, for $g_i = (Ax)_i + \ell_i$,

$$|(x_i + e_i)(g_i + d_i) - x_i g_i| \le |x_i||d_i| + |g_i||e_i| + |e_i||d_i|.$$

Such intermediate CKKS bounds have not been proved uniformly on the box here. The release therefore reports empirical encrypted errors, not a domain-wide noise theorem or a statistical independence claim.

Lean boundary. The release contains a pinned Lean 4/mathlib project. Its checked principal declarations, in namespace `FHEQuadratic`, map directly to the mathematical statements:

Paper claim	Lean declaration
Theorem 1	`quadratic_preservation`
Affine extension	`affine_score_preservation`
Orientation admissibility	`orientation_admissible`
Orientation score equality	`orientation_preservation`
$\lvert C_n\rvert = \lfloor n/2\rfloor + 1$	`canonicalOffsets_card`
Theorem 2, support bound	`orientation_support_upper_bound`
Theorem 3, lower bound	`activeOffsets_card_lower_bound`
Attainment and optimality	`orientation_support_exact`, `orientation_support_optimal`

The integrated `lake build` passed on 19 October 2026 with Lean 4.24.0 and mathlib v4.24.0 (the exact revision is pinned in the dependency lock). The actual axiom-dependency checker passed for all eleven principal declarations: each printed dependency set is exactly `[propext, Classical.choice, Quot.sound]`. These are standard logical axioms; no additional mathematical axiom, `sorryAx`, admitted goal, or kernel-bypass mechanism is used. Exact commands, versions, build results and `#print axioms` output are documented in `docs/FORMAL_VERIFICATION.md`.

`scripts/check_correspondence.py` checks exported matrices using exact rational arithmetic, independently of encrypted timings: 41 matrices and seven public network folds are checked in the release evidence. It connects implementation outputs on those fixtures with the defined orientation formula and admissibility equation, not every future program execution. Formal algebra does not verify the Go evaluator, cryptographic security, compiler/runtime implementation, hardware measurements, or uniform numerical noise. These exclusions remain even after Lean succeeds.

6 Experimental evidence

All historical timing records use an Apple M4 MacBook Air, 16 GiB RAM, Darwin arm64, Go 1.25.3 and Lattigo v6.2.0. The execution thread setting is `GOMAXPROCS=1`; the principal separate-process arm has one Go execution thread per role. Batch, parameters, layout, coefficient model, error requirement and hardware are matched within each comparison. Frozen model labels contain seed numbers such as 20261020; those numbers are reproducibility seeds, not claims of later measurement dates.

Seven models were predetermined in `artifacts/models.json`: an archived integer dimension-8/64-hidden-unit model, synthetic dimension-8/64, dimension-16/128 and dimension-32/128 indefinite scores, held-out dimension-8/96 and dimension-16/96 seeds, and a dimension-16/64 positive-definite score. Synthetic normalization was fixed before benchmarking and applied equally to all methods. Held-out model seeds were not used to choose the mechanism or parameters.

The broad portfolio includes the original packed square network, fused symmetric and ordinary triangular quadratic contraction, direct and official BSGS settings, spectral/LDL affine squares, feature-layout alternatives, and a real conjugate-invariant arm. It contains 336 final model/method records. The fastest tested complete codec-chain baseline is selected per model; this is stronger than comparing with an unfused or untuned network alone. It remains a finite tested portfolio, not proof that every existing implementation has been optimized or outperformed. No complete Orion, HEIR, or other compiler suite was reproduced.

6.1 Principal separate-process measurements

Table 1 comes from five counterbalanced fresh process/key rounds with 31 timed fresh encryptions per plan after separately charged warmup. Each timing is the median of five process medians. RSS is the median of the server child's maximum resident-set value from exit resource usage, not a summed client/server peak. Keys and RSS use decimal MB. The chain includes packing/encoding, encryption, ciphertext serialization, loopback request, server evaluation, response, deserialization and final client decryption/decoding. Evidence-file writes and plaintext reference validation are outside timing.

Using ratios of the displayed marginal medians, dimension-8 oriented direct reduces chain time by 4.6%, server RSS by 24.4%, and keys by 37.5%; dimension-16 oriented direct reduces them by 16.5%, 29.0%, and 43.8%. Dimension-32 oriented BSGS3 reduces them by 14.0%, 18.4%, and 9.1%. These percentages must not be read as paired-round estimates. Paired chain speedup medians are respectively

d	Plan	Server ms	Chain ms	RSS MB	Keys MB	Setup ms
8	triangular_direct	6.188	8.715	86.98	12.586	68.99
8	oriented_direct	5.541	8.310	65.77	7.866	53.19
8	oriented_bsgs3	6.854	9.427	59.79	7.866	52.11
16	triangular_direct	9.429	12.139	127.22	25.172	110.61
16	oriented_direct	7.413	10.139	90.36	14.159	72.27
16	oriented_bsgs3	8.713	11.547	82.49	14.159	74.51
32	symmetric_bsgs3	16.174	19.983	105.43	17.306	171.58
32	oriented_direct	11.937	15.597	135.09	26.745	159.95
32	oriented_bsgs3	13.502	17.181	85.98	15.732	118.79

Table 1: Actual separate client/server processes, $B = 128$. Setup is charged separately from the recurring chain. All plans use identical final parameters and quality requirements.

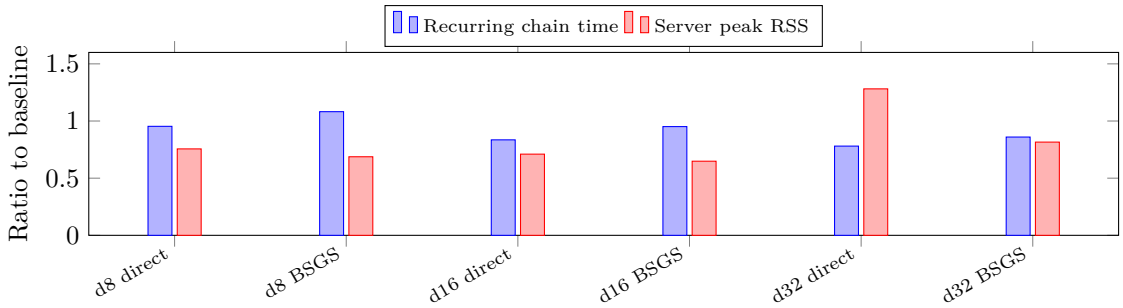


Figure 1: Separate-process marginal-median ratios from raw records. Lower is better. Reducing diagonal support is not a universal time or memory improvement.

1.086, 1.207 and 1.148, with ranges 1.029–1.154, 1.180–1.228 and 1.006–1.175; each selected plan wins all five paired rounds. The median-of-ratios aggregation need not equal the ratio of marginal medians; round-level timing drift may affect either. Five rounds do not establish a hardware-independent performance distribution.

Figure 1 preserves unfavorable results. Dimension-8 BSGS3 is slower on the aggregate chain despite its smaller RSS. Dimension-32 direct is fastest, but its RSS is 135.09 rather than 105.43 MB and its keys are 26.745 rather than 17.306 MB. It is therefore unsuitable as a claimed memory win. The memory-constrained BSGS plan is a different point on the tradeoff curve.

Operation records explain rather than replace measurements. At $n = 8$, direct orientation changes diagonal products from eight to five and total logical rotations from ten to seven; the distinct rotation-key count changes from seven to four. At $n = 16$ the corresponding counts are 16 to nine, 19 to 12, and 15 to eight. The reduction rotations remain charged. At $n = 32$ BSGS schedule structure matters: a smaller diagonal set alone does not predict the key or temporary-memory cost. Every matched packed plan still has one ciphertext product, one relinearization, two rescale levels, and zero bootstraps.

Separate-process setup includes parent public encoding, child startup and configuration transfer, child public encoding, key generation/serialization, HTTP installation, warmup, and child shutdown. The redundant parent/child encoding is charged. Embedded key-deserialization time is not added to its enclosing round-trip twice. Network-to-quadratic folding is separately once-per-public-model. Under this measured loopback lifecycle, both dimension-32 candidates amortize against the baseline at $H = 1$; BSGS3 has lower setup and recurring time. This does not imply the same threshold on a WAN.

6.2 Broader and combined-role evidence

Table 2 reports the seven-model same-machine sweep with nine fresh encryptions after one warmup. Modes ran serially, so it is a breadth/ablation result and not the primary counterbalanced timing estimate. All rows use matched batch 128. The held-out seeds support generalization within this small scalar-quadratic class, not to arbitrary neural workloads.

Model	Baseline	Base ms	Cand. ms	Speedup
Archive 8/64	packed/triangular_direct	6.370	5.836	1.091
Development 8/64	packed/triangular_direct	6.310	5.704	1.106
Development 16/128	packed/triangular_direct	7.844	6.813	1.151
Development 32/128	packed/symmetric_bsgs3	10.621	8.460	1.255
Held-out 8/96	packed/symmetric_direct	6.130	5.652	1.085
Held-out 16/96	packed/symmetric_bsgs3	9.303	8.084	1.151
SPD 16/64	packed/symmetric_bsgs3	8.823	8.311	1.062

Table 2: Broad matched-batch portfolio: fastest tested baseline per frozen model. These are combined-role measurements and must not be mixed with Table 1.

A separate older combined-role HTTP confirmation contains five counterbalanced fresh processes per plan, 31 timed iterations, and actual loopback key/request transport. Its RSS includes client and server in one address space, unlike Table 1. It gives dimension-8 and -16 direct chain reductions of 9.3% and 17.3%; dimension-32 direct gives 19.7% lower chain time but 35.3% higher peak RSS. In that architecture its larger setup amortizes only at $H = 7$, while dimension-32 BSGS3 wins from $H = 1$. These results are retained as controlled ablations, not substituted for isolated server memory or compared in absolute time against the later separated runs. The complete generated table and raw records remain in the artifact.

6.3 Numerical validation and integrity

All 336 final portfolio records pass the fixed 10^{-3} absolute threshold; the worst portfolio error is 1.062×10^{-5} in the conjugate-invariant archived-model arm. Reserved validation adds 560 encrypted batches using 40 fresh key/input seeds per model/method, with 49,000 uniform dyadic input–model pairs and boundary probes. Dimension 8 includes all 256 box corners, and signed basis and fixed boundary patterns are also checked. The maximum observed reserved-input absolute error is $4.24394575 \times 10^{-6}$. In the principal separate-process runs the maximum is 7.345×10^{-8}.

Vectors are reused across same-dimensional models, and outputs within a batch share keys. These are empirical zero-failure statements, not independent pooled trials or a uniform-box theorem. Plaintext neural references and exact coefficient certificates are separately checked. Historical ciphertexts are genuine serialized outputs, but secret keys are not persisted; fresh reruns perform full encryption/decryption rather than pretending the archived ciphertexts can be independently decrypted without those keys.

The release validator reads 685 run records, 986 method records, 2263 serialized ciphertexts and 249088 fixture outputs; these overlapping bookkeeping counts are not additional independent validation trials. It recomputes file integrity and aggregate evidence. Its clean-release checks are recorded separately from the original Apple M4 measurement campaign. Benchmark reruns write new directories and never overwrite the historical measurements. Generated tables and Figure 1 are committed outputs of `scripts/generate_paper_data.py`, and `-check` detects drift.

7 Comparison with prior mechanisms

The closest mathematical comparator is ordinary triangular symmetry elimination. Both choose a scalar-equivalent representative, but the orientation uses the cyclic-offset involution rather than linear index order; this distinction matters to a diagonal evaluator. Fused symmetric contraction is another necessary comparator: network folding can already remove many hidden squares. For the archived model, a separate inertia argument gives a minimum of $\max(p, m) = 5$ products of real linear forms, since the inertia is $(3, 5)$. To justify the general statement, if $q = \sum_{k=1}^{t} L_k R_k$, the common kernel of the L_k has codimension at most t and q vanishes there. It intersects a positive-definite subspace or a negative-definite subspace only at zero, so $t \geq p, m$. Conversely, diagonalize the real symmetric form, pair positive and negative squares using $u^2 - v^2 = (u + v)(u - v)$, and leave any unmatched square as one product. This gives $t = \max(p, m)$. This algebraic cost model makes additions and public transforms free. It is not a 64/5 FHE speedup: at batch 128 the original 64 hidden squares occupy two ciphertexts, while fused contraction has one ciphertext product. Spectral/LDL and complex feature alternatives were executed and retained, but did not dominate the oriented packed plan at the primary workload.

Halevi–Shoup optimize homomorphic linear transforms with diagonal/BSGS and key-switching techniques [7]. We reuse existing kernels and change their input representative under scalar semantics. Coy-

ote [9], Fhelipe [8] and HECO [12] already establish encrypted vectorization, layout selection and compiler restructuring; our work does not invent joint compilation. Orion [5] supplies neural inference packing and scale/bootstrap planning, and HEIR [1] provides a general integration framework. This prototype is not an implemented integration into either system and does not claim a cross-system benchmark win.

Equivalent-expression optimization also predates this work. Q-gym [6] exploits weight repetition through equality saturation. The specific pass here uses a closed-form representative construction and a terminal scalar's equivalence class. nGraph-HE2 [2] provides complex packing and CKKS scheduling optimizations; client-assisted activation variants are outside the present no-intermediate-decryption protocol. These comparisons locate the candidate mechanism more precisely than claiming generic consumer-aware optimization is new.

KeyMemRT [11] is verified prior work with compiler/runtime implementations for rotation-key lifetime management, loading, clearing and prefetch. Eliminating some keys by changing the computation is distinct from managing an unchanged key set, and can coexist with that work. FHE-Agent [13] is a verified Orion-based configuration preprint using an LLM controller. Its reported FHE runtime excludes key generation and its baseline is best-of-ten one-shot configurations; those author claims were not reproduced here. No official implementation was identified in the inspected manuscript, which is a bounded search observation. The present deterministic rewrite and measured lifecycle objective do not rely on an LLM choosing unexplained settings.

The bibliography and `docs/RELATED_WORK.md` record primary-source metadata and implementation links. Our focused search has not established historical priority for the precise cyclic terminal-scalar pass. Publication acceptance, external peer review, or a broad breakthrough are not asserted.

8 Reproducibility and limitations

The release includes the Go backend and dependency lock, Python compiler/checkers, frozen models, measurements, generated material, integrity manifests, and the Lean toolchain/dependency locks. From the repository root, evidence-only checks include:

```
python3 scripts/check_correspondence.py -check
python3 scripts/verify_evidence.py -strict-ciphertexts
python3 scripts/generate_paper_data.py -check
cd lean && lake build
```

The complete clean-checkout instructions, encrypted rerun command, hardware qualifications and LaTeX build are in `REPRODUCING.md`. A rerun on different hardware establishes functioning encryption and accuracy, not reproduction of Apple M4 timing. Historical absolute paths inside raw command records are provenance; supported reproduction commands use the release checkout. Paper/data are licensed CC BY 4.0, and owned code/Lean files MIT; dependencies retain their own licenses.

The measured class is small exact public scalar quadratics with terminal consumers and favorable periodic packing. Multiple outputs, intervening nonlinearities, arbitrary dimensions/batches, and other architectures require separate evaluation. Sparse matrices can invalidate the dense-case optimality hypothesis. The selected plan depends on setup horizon, bandwidth, key memory and runtime memory; support minimization cannot replace profiling. The strongest supported result is an exact, restrictedly optimal representation pass with reproducible encrypted gains and explicit unfavorable cases. Empirical accuracy and published-table security assessment remain the stated evidence levels.

References

[1] Asra Ali, Jaeho Choi, Bryant Gipson, Shruthi Gorantala, Jeremy Kun, Wouter Legiest, Lawrence Lim, Alexander Viand, Meron Zerihun Demissie, and Hongren Zheng. HEIR: A universal compiler for homomorphic encryption. arXiv:2508.11095v1, 2025. URL https://arxiv.org/abs/2508.11095v1.

[2] Fabian Boemer, Anamaria Costache, Rosario Cammarota, and Casimir Wierzynski. nGraph-HE2: A high-throughput framework for neural network inference on encrypted data. arXiv:1908.04172v2, 2019. URL https://arxiv.org/abs/1908.04172v2.

[3] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and

Donggeon Yhee. Security guidelines for implementing homomorphic encryption. *IACR Communications in Cryptology*, 1(4), 2025. doi: 10.62056/anxra69p1. URL https://cic.iacr.org/p/1/4/26.

[4] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. Cryptology ePrint Archive, Paper 2016/421, 2016. URL https://eprint.iacr.org/2016/421. Published in ASIACRYPT 2017.

[5] Austin Ebel, Karthik Garimella, and Brandon Reagen. Orion: A fully homomorphic encryption framework for deep learning. arXiv:2311.03470v3, 2025. URL https://arxiv.org/abs/2311.03470v3. ASPLOS 2025; first preprint 2023.

[6] Cheng Fu, Hanxian Huang, Bram Wasti, Chris Cummins, Riyadh Baghdadi, Kim Hazelwood, Yuandong Tian, Jishen Zhao, and Hugh Leather. Q-gym: An equality saturation framework for DNN inference exploiting weight repetition. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, pages 291–303, 2022. doi: 10.1145/3559009.3569673. URL https://dspace.mit.edu/entities/publication/6e43de1d-b36d-4b18-9999-53e41dbeeea6.

[7] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in HElib. Cryptology ePrint Archive, Paper 2018/244, 2018. URL https://eprint.iacr.org/2018/244. Published in CRYPTO 2018.

[8] Aleksandar Krastev, Nikola Samardzic, Simon Langowski, Srinivas Devadas, and Daniel Sanchez. A tensor compiler with automatic data packing for simple and efficient fully homomorphic encryption. *Proceedings of the ACM on Programming Languages*, 8(PLDI):152:1–152:25, 2024. doi: 10.1145/3656382. URL https://people.csail.mit.edu/devadas/pubs/pldi24_fhelipe.pdf.

[9] Raghav Malik, Kabir Sheth, and Milind Kulkarni. Coyote: A compiler for vectorizing encrypted arithmetic circuits. In *Proceedings of ASPLOS*, pages 118–133, 2023. URL https://raghav198.github.io/. Official implementation: https://github.com/raghav198/coyote.

[10] Tune Insight. Lattigo v6.2.0. Software, 2026. URL https://github.com/tuneinsight/lattigo/tree/v6.2.0. Pinned implementation; accessed 2026-10-19.

[11] Eymen Ünay, Björn Franke, and Jackson Woodruff. KeyMemRT compiler and runtime: Unlocking memory-scalable FHE. arXiv:2601.18445v1, 2026. URL https://arxiv.org/abs/2601.18445v1.

[12] Alexander Viand, Patrick Jattke, Miro Haller, and Anwar Hithnawi. HECO: Fully homomorphic encryption compiler. In *32nd USENIX Security Symposium*, pages 4715–4732. USENIX Association, 2023. URL https://www.usenix.org/conference/usenixsecurity23/presentation/viand.

[13] Nuo Xu, Zhaoting Gong, Ran Ran, Jinwei Tang, Wujie Wen, and Caiwen Ding. FHE-Agent: Automating CKKS configuration for practical encrypted inference via an LLM-guided agentic framework. arXiv:2511.18653v1, 2025. URL https://arxiv.org/abs/2511.18653v1.